The Red Hat newlib C Math Library

`libm` 2.5.0
December 2016

Steve Chamberlain
Roland Pesch
Red Hat Support
Jeff Johnston

Red Hat Support
sac@cygnus.com
pesch@cygnus.com
jjohnstn@redhat.com

Copyright © 1992, 1993, 1994-2004 Red Hat, Inc.

`libm` includes software developed at SunPro, a Sun Microsystems, Inc. business. Permission to use, copy, modify, and distribute this software is freely granted, provided that this notice is preserved.

Permission is granted to make and distribute verbatim copies of this manual provided the copyright notice and this permission notice are preserved on all copies.

Permission is granted to copy and distribute modified versions of this manual under the conditions for verbatim copying, subject to the terms of the GNU General Public License, which includes the provision that the entire resulting derived work is distributed under the terms of a permission notice identical to this one.

Permission is granted to copy and distribute translations of this manual into another language, under the above conditions for modified versions.

1 Mathematical Functions (`math.h`)

This chapter groups a wide variety of mathematical functions. The corresponding definitions and declarations are in `math.h`. The definition of HUGE_VAL from `math.h` is of particular interest.

1. The representation of infinity as a `double` is defined as `HUGE_VAL`; this number is returned on overflow by many functions. The macro `HUGE_VALF` is a corresponding value for `float`.

Alternative declarations of the mathematical functions, which exploit specific machine capabilities to operate faster—but generally have less error checking and may reflect additional limitations on some machines—are available when you include `fastmath.h` instead of `math.h`.

1.1 Error Handling

There are two different versions of the math library routines: IEEE and POSIX. The version may be selected at runtime by setting the global variable `_LIB_VERSION`, defined in `math.h`. It may be set to one of the following constants defined in `math.h`: `_IEEE_` or `_POSIX_`. The `_LIB_VERSION` variable is not specific to any thread, and changing it will affect all threads.

The versions of the library differ only in the setting of `errno`.

In IEEE mode, `errno` is never set.

In POSIX mode, `errno` is set correctly.

The library is set to IEEE mode by default.

The majority of the floating-point math functions are written so as to produce the floating-point exceptions (e.g. "invalid", "divide-by-zero") as required by the C and POSIX standards, for floating-point implementations that support them. Newlib does not provide the floating-point exception access routines defined in the standards for fenv.h, though, which is why they are considered unsupported. It is mentioned in case you have separately-provided access routines so that you are aware that they can be caused.

1.2 Standards Compliance And Portability

Most of the individual function descriptions describe the standards to which each function complies. However, these descriptions are mostly out of date, having been written before C99 was released. One of these days we'll get around to updating the rest of them. (If you'd like to help, please let us know.)

"C99" refers to ISO/IEC 9899:1999, "Programming languages–C". "POSIX" refers to IEEE Standard 1003.1. POSIX® is a registered trademark of The IEEE.

1.3 acos, acosf—arc cosine

Synopsis

```
#include <math.h>
double acos(double x);
float acosf(float x);
```

Description

`acos` computes the inverse cosine (arc cosine) of the input value. Arguments to `acos` must be in the range -1 to 1.

`acosf` is identical to `acos`, except that it performs its calculations on `floats`.

Returns

`acos` and `acosf` return values in radians, in the range of `0` to π.

If x is not between -1 and 1, the returned value is NaN (not a number), and the global variable `errno` is set to `EDOM`.

1.4 acosh, acoshf—inverse hyperbolic cosine

Synopsis

```
#include <math.h>
double acosh(double x);
float acoshf(float x);
```

Description

`acosh` calculates the inverse hyperbolic cosine of x. `acosh` is defined as

$$ln\left(x + \sqrt{x^2 - 1}\right)$$

x must be a number greater than or equal to 1.

`acoshf` is identical, other than taking and returning floats.

Returns

`acosh` and `acoshf` return the calculated value. If x less than 1, the return value is NaN and `errno` is set to `EDOM`.

Portability

Neither `acosh` nor `acoshf` are ANSI C. They are not recommended for portable programs.

1.5 `asin`, `asinf`—arc sine

Synopsis

```
#include <math.h>
double asin(double x);
float asinf(float x);
```

Description

`asin` computes the inverse sine (arc sine) of the argument x. Arguments to `asin` must be in the range -1 to 1.

`asinf` is identical to `asin`, other than taking and returning floats.

Returns

`asin` returns values in radians, in the range of $-\pi/2$ to $\pi/2$.

If x is not in the range -1 to 1, `asin` and `asinf` return NaN (not a number), and the global variable `errno` is set to `EDOM`.

1.6 `asinh`, `asinhf`—inverse hyperbolic sine

Synopsis

```
#include <math.h>
double asinh(double x);
float asinhf(float x);
```

Description
`asinh` calculates the inverse hyperbolic sine of x. `asinh` is defined as

$$sign(x) \times ln\Big(|x| + \sqrt{1 + x^2}\Big)$$

`asinhf` is identical, other than taking and returning floats.

Returns
`asinh` and `asinhf` return the calculated value.

Portability
Neither `asinh` nor `asinhf` are ANSI C.

1.7 `atan`, `atanf`—arc tangent

Synopsis

```
#include <math.h>
double atan(double x);
float atanf(float x);
```

Description

`atan` computes the inverse tangent (arc tangent) of the input value.

`atanf` is identical to `atan`, save that it operates on `floats`.

Returns

`atan` returns a value in radians, in the range of $-\pi/2$ to $\pi/2$.

Portability

`atan` is ANSI C. `atanf` is an extension.

1.8 atan2, atan2f—arc tangent of y/x

Synopsis

```
#include <math.h>
double atan2(double y,double x);
float atan2f(float y,float x);
```

Description

`atan2` computes the inverse tangent (arc tangent) of y/x. `atan2` produces the correct result even for angles near $\pi/2$ or $-\pi/2$ (that is, when x is near 0).

`atan2f` is identical to `atan2`, save that it takes and returns `float`.

Returns

`atan2` and `atan2f` return a value in radians, in the range of $-\pi$ to π.

Portability

`atan2` is ANSI C. `atan2f` is an extension.

1.9 atanh, atanhf—inverse hyperbolic tangent

Synopsis

```
#include <math.h>
double atanh(double x);
float atanhf(float x);
```

Description

`atanh` calculates the inverse hyperbolic tangent of x.

`atanhf` is identical, other than taking and returning `float` values.

Returns

`atanh` and `atanhf` return the calculated value.

If $|x|$ is greater than 1, the global `errno` is set to `EDOM` and the result is a NaN. A `DOMAIN error` is reported.

If $|x|$ is 1, the global `errno` is set to `EDOM`; and the result is infinity with the same sign as `x`. A `SING error` is reported.

Portability

Neither `atanh` nor `atanhf` are ANSI C.

1.10 `jN`, `jNf`, `yN`, `yNf`—Bessel functions

Synopsis

```
#include <math.h>
double j0(double x);
float j0f(float x);
double j1(double x);
float j1f(float x);
double jn(int n, double x);
float jnf(int n, float x);
double y0(double x);
float y0f(float x);
double y1(double x);
float y1f(float x);
double yn(int n, double x);
float ynf(int n, float x);
```

Description

The Bessel functions are a family of functions that solve the differential equation

$$x^2 \frac{d^2y}{dx^2} + x\frac{dy}{dx} + (x^2 - p^2)y = 0$$

These functions have many applications in engineering and physics.

`jn` calculates the Bessel function of the first kind of order n. `j0` and `j1` are special cases for order 0 and order 1 respectively.

Similarly, `yn` calculates the Bessel function of the second kind of order n, and `y0` and `y1` are special cases for order 0 and 1.

`jnf`, `j0f`, `j1f`, `ynf`, `y0f`, and `y1f` perform the same calculations, but on `float` rather than `double` values.

Returns

The value of each Bessel function at x is returned.

Portability

None of the Bessel functions are in ANSI C.

1.11 cbrt, cbrtf—cube root

Synopsis

```
#include <math.h>
double cbrt(double x);
float  cbrtf(float x);
```

Description
`cbrt` computes the cube root of the argument.

Returns
The cube root is returned.

Portability
`cbrt` is in System V release 4. `cbrtf` is an extension.

1.12 `copysign`, `copysignf`—sign of *y*, magnitude of *x*

Synopsis

```
#include <math.h>
double copysign (double x, double y);
float copysignf (float x, float y);
```

Description

`copysign` constructs a number with the magnitude (absolute value) of its first argument, *x*, and the sign of its second argument, *y*.

`copysignf` does the same thing; the two functions differ only in the type of their arguments and result.

Returns

`copysign` returns a `double` with the magnitude of *x* and the sign of *y*. `copysignf` returns a `float` with the magnitude of *x* and the sign of *y*.

Portability

`copysign` is not required by either ANSI C or the System V Interface Definition (Issue 2).

1.13 `cosh`, `coshf`—hyperbolic cosine

Synopsis

```
#include <math.h>
double cosh(double x);
float coshf(float x);
```

Description

`cosh` computes the hyperbolic cosine of the argument x. `cosh(x)` is defined as

$$\frac{(e^x + e^{-x})}{2}$$

Angles are specified in radians. `coshf` is identical, save that it takes and returns `float`.

Returns
The computed value is returned. When the correct value would create an overflow, `cosh` returns the value `HUGE_VAL` with the appropriate sign, and the global value `errno` is set to `ERANGE`.

Portability
`cosh` is ANSI. `coshf` is an extension.

1.14 `erf`, `erff`, `erfc`, `erfcf`—error function

Synopsis

```
#include <math.h>
double erf(double x);
float erff(float x);
double erfc(double x);
float erfcf(float x);
```

Description

`erf` calculates an approximation to the "error function", which estimates the probability that an observation will fall within x standard deviations of the mean (assuming a normal distribution). The error function is defined as

$$\frac{2}{\sqrt{\pi}} \times \int_0^x e^{-t^2} dt$$

`erfc` calculates the complementary probability; that is, `erfc(x)` is `1 - erf(x)`. `erfc` is computed directly, so that you can use it to avoid the loss of precision that would result from subtracting large probabilities (on large x) from 1.

`erff` and `erfcf` differ from `erf` and `erfc` only in the argument and result types.

Returns

For positive arguments, `erf` and all its variants return a probability—a number between 0 and 1.

Portability

None of the variants of `erf` are ANSI C.

1.15 exp, expf—exponential

Synopsis

```
#include <math.h>
double exp(double x);
float expf(float x);
```

Description

exp and expf calculate the exponential of x, that is, e^x (where e is the base of the natural system of logarithms, approximately 2.71828).

Returns

On success, exp and expf return the calculated value. If the result underflows, the returned value is 0. If the result overflows, the returned value is HUGE_VAL. In either case, errno is set to ERANGE.

Portability

exp is ANSI C. expf is an extension.

1.16 `exp10`, `exp10f`—exponential, base 10

Synopsis

```
#include <math.h>
double exp10(double x);
float exp10f(float x);
```

Description

`exp10` and `exp10f` calculate 10 ^ x, that is, 10^x

Returns

On success, `exp10` and `exp10f` return the calculated value. If the result underflows, the returned value is `0`. If the result overflows, the returned value is `HUGE_VAL`. In either case, `errno` is set to `ERANGE`.

Portability

`exp10` and `exp10f` are GNU extensions.

1.17 exp2, exp2f—exponential, base 2

Synopsis

```
#include <math.h>
double exp2(double x);
float exp2f(float x);
```

Description

exp2 and exp2f calculate 2 ^ x, that is, 2^x

Returns

On success, exp2 and exp2f return the calculated value. If the result underflows, the returned value is 0. If the result overflows, the returned value is HUGE_VAL. In either case, errno is set to ERANGE.

Portability

ANSI C, POSIX.

1.18 `expm1`, `expm1f`—exponential minus 1

Synopsis

```
#include <math.h>
double expm1(double x);
float expm1f(float x);
```

Description

`expm1` and `expm1f` calculate the exponential of x and subtract 1, that is, $e^x - 1$ (where e is the base of the natural system of logarithms, approximately 2.71828). The result is accurate even for small values of x, where using `exp(x)-1` would lose many significant digits.

Returns

e raised to the power x, minus 1.

Portability

Neither `expm1` nor `expm1f` is required by ANSI C or by the System V Interface Definition (Issue 2).

1.19 `fabs`, `fabsf`—absolute value (magnitude)

Synopsis

```
#include <math.h>
double fabs(double x);
float fabsf(float x);
```

Description

`fabs` and `fabsf` calculate $|x|$, the absolute value (magnitude) of the argument x, by direct manipulation of the bit representation of x.

Returns

The calculated value is returned. No errors are detected.

Portability

`fabs` is ANSI. `fabsf` is an extension.

1.20 `fdim`, `fdimf`—positive difference

Synopsis

```
#include <math.h>
double fdim(double x, double y);
float fdimf(float x, float y);
```

Description

The `fdim` functions determine the positive difference between their arguments, returning:

`x - y if x > y, or`

`+0 if` $x \leq y$`, or`

`NAN if either argument is NAN.`

A range error may occur.

Returns

The `fdim` functions return the positive difference value.

Portability

ANSI C, POSIX.

1.21 `floor`, `floorf`, `ceil`, `ceilf`—floor and ceiling

Synopsis

```
#include <math.h>
double floor(double x);
float floorf(float x);
double ceil(double x);
float ceilf(float x);
```

Description

`floor` and `floorf` find $\lfloor x \rfloor$, the nearest integer less than or equal to x. `ceil` and `ceilf` find $\lceil x \rceil$, the nearest integer greater than or equal to x.

Returns

`floor` and `ceil` return the integer result as a double. `floorf` and `ceilf` return the integer result as a float.

Portability

`floor` and `ceil` are ANSI. `floorf` and `ceilf` are extensions.

1.22 `fma`, `fmaf`—floating multiply add

Synopsis

```
#include <math.h>
double fma(double x, double y, double z);
float fmaf(float x, float y, float z);
```

Description

The `fma` functions compute (*x* * *y*) + *z*, rounded as one ternary operation: they compute the value (as if) to infinite precision and round once to the result format, according to the rounding mode characterized by the value of FLT_ROUNDS. That is, they are supposed to do this: see below.

Returns

The `fma` functions return (*x* * *y*) + *z*, rounded as one ternary operation.

Bugs

This implementation does not provide the function that it should, purely returning "(*x* * *y*) + *z*;" with no attempt at all to provide the simulated infinite precision intermediates which are required. DO NOT USE THEM.

If double has enough more precision than float, then `fmaf` should provide the expected numeric results, as it does use double for the calculation. But since this is not the case for all platforms, this manual cannot determine if it is so for your case.

Portability

ANSI C, POSIX.

1.23 fmax, fmaxf—maximum

Synopsis

```
#include <math.h>
double fmax(double x, double y);
float fmaxf(float x, float y);
```

Description
The fmax functions determine the maximum numeric value of their arguments. NaN arguments are treated as missing data: if one argument is a NaN and the other numeric, then the fmax functions choose the numeric value.

Returns
The fmax functions return the maximum numeric value of their arguments.

Portability
ANSI C, POSIX.

1.24 `fmin`, `fminf`—minimum

Synopsis

```
#include <math.h>
double fmin(double x, double y);
float fminf(float x, float y);
```

Description
The `fmin` functions determine the minimum numeric value of their arguments. NaN arguments are treated as missing data: if one argument is a NaN and the other numeric, then the `fmin` functions choose the numeric value.

Returns
The `fmin` functions return the minimum numeric value of their arguments.

Portability
ANSI C, POSIX.

1.25 `fmod`, `fmodf`—floating-point remainder (modulo)

Synopsis

```
#include <math.h>
double fmod(double x, double y);
float fmodf(float x, float y);
```

Description

The `fmod` and `fmodf` functions compute the floating-point remainder of x/y (x modulo y).

Returns

The `fmod` function returns the value $x - i \times y$, for the largest integer i such that, if y is nonzero, the result has the same sign as x and magnitude less than the magnitude of y.

`fmod(x,0)` returns NaN, and sets `errno` to `EDOM`.

Portability

`fmod` is ANSI C. `fmodf` is an extension.

1.26 `frexp`, `frexpf`—split floating-point number

Synopsis

```
#include <math.h>
double frexp(double val, int *exp);
float frexpf(float val, int *exp);
```

Description

All nonzero, normal numbers can be described as *m* * 2**p*. `frexp` represents the double *val* as a mantissa *m* and a power of two *p*. The resulting mantissa will always be greater than or equal to `0.5`, and less than `1.0` (as long as *val* is nonzero). The power of two will be stored in `*`*exp*.

m and *p* are calculated so that $val = m \times 2^p$.

`frexpf` is identical, other than taking and returning floats rather than doubles.

Returns

`frexp` returns the mantissa *m*. If *val* is `0`, infinity, or Nan, `frexp` will set `*`*exp* to `0` and return *val*.

Portability

`frexp` is ANSI. `frexpf` is an extension.

1.27 gamma, gammaf, lgamma, lgammaf, gamma_r, gammaf_r, lgamma_r, lgammaf_r, tgamma, and tgammaf—logarithmic and plain gamma functions

Synopsis

```
#include <math.h>
double gamma(double x);
float gammaf(float x);
double lgamma(double x);
float lgammaf(float x);
double gamma_r(double x, int *signgamp);
float gammaf_r(float x, int *signgamp);
double lgamma_r(double x, int *signgamp);
float lgammaf_r(float x, int *signgamp);
double tgamma(double x);
float tgammaf(float x);
```

Description

`gamma` calculates $ln(\Gamma(x))$, the natural logarithm of the gamma function of x. The gamma function (`exp(gamma(x))`) is a generalization of factorial, and retains the property that $\Gamma(N) \equiv N \times \Gamma(N-1)$. Accordingly, the results of the gamma function itself grow very quickly. `gamma` is defined as $ln(\Gamma(x))$ rather than simply $\Gamma(x)$ to extend the useful range of results representable.

The sign of the result is returned in the global variable `signgam`, which is declared in math.h.

`gammaf` performs the same calculation as `gamma`, but uses and returns `float` values.

`lgamma` and `lgammaf` are alternate names for `gamma` and `gammaf`. The use of `lgamma` instead of `gamma` is a reminder that these functions compute the log of the gamma function, rather than the gamma function itself.

The functions `gamma_r`, `gammaf_r`, `lgamma_r`, and `lgammaf_r` are just like `gamma`, `gammaf`, `lgamma`, and `lgammaf`, respectively, but take an additional argument. This additional argument is a pointer to an integer. This additional argument is used to return the sign of the result, and the global variable `signgam` is not used. These functions may be used for reentrant calls (but they will still set the global variable `errno` if an error occurs).

`tgamma` and `tgammaf` are the "true gamma" functions, returning $\Gamma(x)$, the gamma function of x–without a logarithm. (They are apparently so named because of the prior existence of the old, poorly-named `gamma` functions which returned the log of gamma up through BSD 4.2.)

Returns

Normally, the computed result is returned.

When x is a nonpositive integer, `gamma` returns `HUGE_VAL` and `errno` is set to `EDOM`. If the result overflows, `gamma` returns `HUGE_VAL` and `errno` is set to `ERANGE`.

Portability

Neither `gamma` nor `gammaf` is ANSI C. It is better not to use either of these; use `lgamma` or `tgamma` instead.

`lgamma`, `lgammaf`, `tgamma`, and `tgammaf` are nominally C standard in terms of the base return values, although the *signgam* global for `lgamma` is not standard.

1.28 hypot, hypotf—distance from origin

Synopsis

```
#include <math.h>
double hypot(double x, double y);
float hypotf(float x, float y);
```

Description

`hypot` calculates the Euclidean distance $\sqrt{x^2 + y^2}$ between the origin (0,0) and a point represented by the Cartesian coordinates (x,y). `hypotf` differs only in the type of its arguments and result.

Returns

Normally, the distance value is returned. On overflow, `hypot` returns `HUGE_VAL` and sets `errno` to `ERANGE`.

Portability

`hypot` and `hypotf` are not ANSI C.

1.29 `ilogb`, `ilogbf`—get exponent of floating-point number

Synopsis

```
#include <math.h>
int ilogb(double val);
int ilogbf(float val);
```

Description

All nonzero, normal numbers can be described as *m* * 2**p*. `ilogb` and `ilogbf` examine the argument *val*, and return *p*. The functions `frexp` and `frexpf` are similar to `ilogb` and `ilogbf`, but also return *m*.

Returns

`ilogb` and `ilogbf` return the power of two used to form the floating-point argument. If *val* is `0`, they return `FP_ILOGB0`. If *val* is infinite, they return `INT_MAX`. If *val* is NaN, they return `FP_ILOGBNAN`. (`FP_ILOGB0` and `FP_ILOGBNAN` are defined in math.h, but in turn are defined as INT_MIN or INT_MAX from limits.h. The value of FP_ILOGB0 may be either INT_MIN or -INT_MAX. The value of FP_ILOGBNAN may be either INT_MAX or INT_MIN.)

Portability
C99, POSIX

1.30 infinity, infinityf—representation of infinity

Synopsis

```
#include <math.h>
double infinity(void);
float infinityf(void);
```

Description

infinity and infinityf return the special number IEEE infinity in double- and single-precision arithmetic respectively.

Portability

infinity and infinityf are neither standard C nor POSIX. C and POSIX require macros HUGE_VAL and HUGE_VALF to be defined in math.h, which Newlib defines to be infinities corresponding to these archaic infinity() and infinityf() functions in floating-point implementations which do have infinities.

1.31 isgreater, isgreaterequal, isless, islessequal, islessgreater, and isunordered—comparison macros

Synopsis

```
#include <math.h>
int isgreater(real-floating x, real-floating y);
int isgreaterequal(real-floating x, real-floating y);
int isless(real-floating x, real-floating y);
int islessequal(real-floating x, real-floating y);
int islessgreater(real-floating x, real-floating y);
int isunordered(real-floating x, real-floating y);
```

Description

`isgreater`, `isgreaterequal`, `isless`, `islessequal`, `islessgreater`, and `isunordered` are macros defined for use in comparing floating-point numbers without raising any floating-point exceptions.

The relational operators (i.e. `<`, `>`, `<=`, and `>=`) support the usual mathematical relationships between numeric values. For any ordered pair of numeric values exactly one of the relationships–less, greater, and equal–is true. Relational operators may raise the "invalid" floating-point exception when argument values are NaNs. For a NaN and a numeric value, or for two NaNs, just the unordered relationship is true (i.e., if one or both of the arguments a NaN, the relationship is called unordered). The specified macros are quiet (non floating-point exception raising) versions of the relational operators, and other comparison macros that facilitate writing efficient code that accounts for NaNs without suffering the "invalid" floating-point exception. In the synopses shown, "real-floating" indicates that the argument is an expression of real floating type.

Please note that saying that the macros do not raise floating-point exceptions, it is referring to the function that they are performing. It is certainly possible to give them an expression which causes an exception. For example:

`NaN < 1.0` causes an "invalid" exception,

`isless(NaN, 1.0)`
does not, and

`isless(NaN*0., 1.0)`
causes an exception due to the "NaN*0.", but not from the resultant reduced comparison of isless(NaN, 1.0).

Returns

No floating-point exceptions are raised for any of the macros.
The `isgreater` macro returns the value of (x) > (y).
The `isgreaterequal` macro returns the value of (x) >= (y).
The `isless` macro returns the value of (x) < (y).
The `islessequal` macro returns the value of (x) <= (y).
The `islessgreater` macro returns the value of (x) < (y) || (x) > (y).
The `isunordered` macro returns 1 if either of its arguments is NaN and 0 otherwise.

Portability
C99, POSIX.

1.32 `fpclassify`, `isfinite`, `isinf`, `isnan`, and `isnormal`—floating-point classification macros; `finite`, `finitef`, `isinf`, `isinff`, `isnan`, `isnanf`—test for exceptional numbers

Synopsis

```
[C99 standard macros:]
#include <math.h>
int fpclassify(real-floating x);
int isfinite(real-floating x);
int isinf(real-floating x);
int isnan(real-floating x);
int isnormal(real-floating x);

[Archaic SUSv2 functions:]
#include <math.h>
int isnan(double arg);
int isinf(double arg);
int finite(double arg);
int isnanf(float arg);
int isinff(float arg);
int finitef(float arg);
```

Description

`fpclassify`, `isfinite`, `isinf`, `isnan`, and `isnormal` are macros defined for use in classifying floating-point numbers. This is a help because of special "values" like NaN and infinities. In the synopses shown, "real-floating" indicates that the argument is an expression of real floating type. These function-like macros are C99 and POSIX-compliant, and should be used instead of the now-archaic SUSv2 functions.

The `fpclassify` macro classifies its argument value as NaN, infinite, normal, subnormal, zero, or into another implementation-defined category. First, an argument represented in a format wider than its semantic type is converted to its semantic type. Then classification is based on the type of the argument. The `fpclassify` macro returns the value of the number classification macro appropriate to the value of its argument:

`FP_INFINITE`
: x is either plus or minus infinity;

`FP_NAN`
: x is "Not A Number" (plus or minus);

`FP_NORMAL`
: x is a "normal" number (i.e. is none of the other special forms);

`FP_SUBNORMAL`
: x is too small be stored as a regular normalized number (i.e. loss of precision is likely); or

`FP_ZERO`
: x is 0 (either plus or minus).

The "`is`" set of macros provide a useful set of shorthand ways for classifying floating-point numbers, providing the following equivalent relations:

`isfinite(x)`
: returns non-zero if x is finite. (It is equivalent to (`fpclassify`(x) != FP_INFINITE && `fpclassify`(x) != FP_NAN).)

`isinf(x)`
: returns non-zero if x is infinite. (It is equivalent to (`fpclassify`(x) == FP_INFINITE).)

`isnan(x)`
: returns non-zero if x is NaN. (It is equivalent to (`fpclassify`(x) == FP_NAN).)

`isnormal(x)`
: returns non-zero if x is normal. (It is equivalent to (`fpclassify`(x) == FP_NORMAL).)

The archaic SUSv2 functions provide information on the floating-point argument supplied.

There are five major number formats ("exponent" referring to the biased exponent in the binary-encoded number):

`zero`
: A number which contains all zero bits, excluding the sign bit.

`subnormal`
: A number with a zero exponent but a nonzero fraction.

`normal`
: A number with an exponent and a fraction.

`infinity`
: A number with an all 1's exponent and a zero fraction.

`NAN`
: A number with an all 1's exponent and a nonzero fraction.

`isnan` returns 1 if the argument is a nan. `isinf` returns 1 if the argument is infinity. `finite` returns 1 if the argument is zero, subnormal or normal. The `isnanf`, `isinff` and `finitef` functions perform the same operations as their `isnan`, `isinf` and `finite` counterparts, but on single-precision floating-point numbers.

It should be noted that the C99 standard dictates that `isnan` and `isinf` are macros that operate on multiple types of floating-point. The SUSv2 standard declares `isnan` as a function taking double. Newlib has decided to declare them both as functions and as macros in math.h to maintain backward compatibility.

Returns

The fpclassify macro returns the value corresponding to the appropriate FP_ macro.
The isfinite macro returns nonzero if x is finite, else 0.
The isinf macro returns nonzero if x is infinite, else 0.
The isnan macro returns nonzero if x is an NaN, else 0.
The isnormal macro returns nonzero if x has a normal value, else 0.

Portability

math.h macros are C99, POSIX.1-2001.

The functions originate from BSD; isnan was listed in the X/Open Portability Guide and Single Unix Specification, but was dropped when the macro was standardized in POSIX.1-2001.

1.33 `ldexp`, `ldexpf`—load exponent

Synopsis

```
#include <math.h>
double ldexp(double val, int exp);
float ldexpf(float val, int exp);
```

Description

`ldexp` calculates the value $val \times 2^{exp}$. `ldexpf` is identical, save that it takes and returns `float` rather than `double` values.

Returns

`ldexp` returns the calculated value.

Underflow and overflow both set `errno` to `ERANGE`. On underflow, `ldexp` and `ldexpf` return 0.0. On overflow, `ldexp` returns plus or minus `HUGE_VAL`.

Portability

`ldexp` is ANSI. `ldexpf` is an extension.

1.34 log, logf—natural logarithms

Synopsis

```
#include <math.h>
double log(double x);
float logf(float x);
```

Description

Return the natural logarithm of *x*, that is, its logarithm base e (where e is the base of the natural system of logarithms, 2.71828...). `log` and `logf` are identical save for the return and argument types.

Returns

Normally, returns the calculated value. When *x* is zero, the returned value is `-HUGE_VAL` and `errno` is set to `ERANGE`. When *x* is negative, the returned value is NaN (not a number) and `errno` is set to `EDOM`.

Portability

`log` is ANSI. `logf` is an extension.

1.35 `log10`, `log10f`—base 10 logarithms

Synopsis

```
#include <math.h>
double log10(double x);
float log10f(float x);
```

Description

`log10` returns the base 10 logarithm of x. It is implemented as `log(x) / log(10)`.

`log10f` is identical, save that it takes and returns `float` values.

Returns

`log10` and `log10f` return the calculated value.

See the description of `log` for information on errors.

Portability

`log10` is ANSI C. `log10f` is an extension.

1.36 log1p, log1pf—log of 1 + x

Synopsis

```
#include <math.h>
double log1p(double x);
float log1pf(float x);
```

Description

`log1p` calculates $ln(1 + x)$, the natural logarithm of `1+x`. You can use `log1p` rather than '`log(1+x)`' for greater precision when x is very small.

`log1pf` calculates the same thing, but accepts and returns `float` values rather than `double`.

Returns

`log1p` returns a `double`, the natural log of `1+x`. `log1pf` returns a `float`, the natural log of `1+x`.

Portability

Neither `log1p` nor `log1pf` is required by ANSI C or by the System V Interface Definition (Issue 2).

1.37 log2, log2f—base 2 logarithm

Synopsis

```
#include <math.h>
double log2(double x);
float log2f(float x);
```

Description

The log2 functions compute the base-2 logarithm of x. A domain error occurs if the argument is less than zero. A range error occurs if the argument is zero.

The Newlib implementations are not full, intrinisic calculations, but rather are derivatives based on log. (Accuracy might be slightly off from a direct calculation.) In addition to functions, they are also implemented as macros defined in math.h:

```
#define log2(x) (log (x) / _M_LN2)
#define log2f(x) (logf (x) / (float) _M_LN2)
```

To use the functions instead, just undefine the macros first.

Returns

The log2 functions return $log_2(x)$ on success. When x is zero, the returned value is -HUGE_VAL and errno is set to ERANGE. When x is negative, the returned value is NaN (not a number) and errno is set to EDOM.

Portability

C99, POSIX, System V Interface Definition (Issue 6).

1.38 logb, logbf—get exponent of floating-point number

Synopsis

```
#include <math.h>
double logb(double x);
float logbf(float x);
```

Description

The logb functions extract the exponent of x, as a signed integer value in floating-point format. If x is subnormal it is treated as though it were normalized; thus, for positive finite x, $1 \leq (x \cdot FLT_RADIX^{-logb(x)}) < FLT_RADIX$. A domain error may occur if the argument is zero. In this floating-point implementation, FLT_RADIX is 2. Which also means that for finite x, logb(x) = floor(log2(fabs(x))).

All nonzero, normal numbers can be described as $m \cdot 2^p$, where $1.0 \leq m < 2.0$. The logb functions examine the argument x, and return p. The frexp functions are similar to the logb functions, but returning m adjusted to the interval [.5, 1) or 0, and p+1.

Returns

When x is:
+inf or -inf, +inf is returned;
NaN, NaN is returned;
0, -inf is returned, and the divide-by-zero exception is raised;
otherwise, the logb functions return the signed exponent of x.

Portability

ANSI C, POSIX

See Also

frexp, ilogb

1.39 `lrint`, `lrintf`, `llrint`, `llrintf`—round to integer

Synopsis

```
#include <math.h>
long int lrint(double x);
long int lrintf(float x);
long long int llrint(double x);
long long int llrintf(float x);
```

Description

The `lrint` and `llrint` functions round their argument to the nearest integer value, using the current rounding direction. If the rounded value is outside the range of the return type, the numeric result is unspecified. A range error may occur if the magnitude of x is too large. The "inexact" floating-point exception is raised in implementations that support it when the result differs in value from the argument (i.e., when a fraction actually has been truncated).

Returns

x rounded to an integral value, using the current rounding direction.

See Also

`lround`

Portability

ANSI C, POSIX

1.40 `lround`, `lroundf`, `llround`, `llroundf`—round to integer, to nearest

Synopsis

```
#include <math.h>
long int lround(double x);
long int lroundf(float x);
long long int llround(double x);
long long int llroundf(float x);
```

Description
The `lround` and `llround` functions round their argument to the nearest integer value, rounding halfway cases away from zero, regardless of the current rounding direction. If the rounded value is outside the range of the return type, the numeric result is unspecified (depending upon the floating-point implementation, not the library). A range error may occur if the magnitude of x is too large.

Returns
x rounded to an integral value as an integer.

See Also
See the `round` functions for the return being the same floating-point type as the argument. `lrint`, `llrint`.

Portability
ANSI C, POSIX

1.41 `modf`, `modff`—split fractional and integer parts

Synopsis

```
#include <math.h>
double modf(double val, double *ipart);
float modff(float val, float *ipart);
```

Description

`modf` splits the double *val* apart into an integer part and a fractional part, returning the fractional part and storing the integer part in `*ipart`. No rounding whatsoever is done; the sum of the integer and fractional parts is guaranteed to be exactly equal to *val*. That is, if *realpart* = modf(*val*, *&intpart*); then '`realpart+intpart`' is the same as *val*. `modff` is identical, save that it takes and returns `float` rather than `double` values.

Returns

The fractional part is returned. Each result has the same sign as the supplied argument *val*.

Portability

`modf` is ANSI C. `modff` is an extension.

1.42 nan, nanf—representation of "Not a Number"

Synopsis

```
#include <math.h>
double nan(const char *unused);
float nanf(const char *unused);
```

Description

`nan` and `nanf` return an IEEE NaN (Not a Number) in double- and single-precision arithmetic respectively. The argument is currently disregarded.

1.43 `nearbyint`, `nearbyintf`—round to integer

Synopsis

```
#include <math.h>
double nearbyint(double x);
float nearbyintf(float x);
```

Description
The `nearbyint` functions round their argument to an integer value in floating-point format, using the current rounding direction and (supposedly) without raising the "inexact" floating-point exception. See the `rint` functions for the same function with the "inexact" floating-point exception being raised when appropriate.

Bugs
Newlib does not support the floating-point exception model, so that the floating-point exception control is not present and thereby what may be seen will be compiler and hardware dependent in this regard. The Newlib `nearbyint` functions are identical to the `rint` functions with respect to the floating-point exception behavior, and will cause the "inexact" exception to be raised for most targets.

Returns
x rounded to an integral value, using the current rounding direction.

Portability
ANSI C, POSIX

See Also
`rint`, `round`

1.44 nextafter, nextafterf—get next number

Synopsis

```
#include <math.h>
double nextafter(double val, double dir);
float nextafterf(float val, float dir);
```

Description

`nextafter` returns the double-precision floating-point number closest to *val* in the direction toward *dir*. `nextafterf` performs the same operation in single precision. For example, `nextafter(0.0,1.0)` returns the smallest positive number which is representable in double precision.

Returns

Returns the next closest number to *val* in the direction toward *dir*.

Portability

Neither `nextafter` nor `nextafterf` is required by ANSI C or by the System V Interface Definition (Issue 2).

1.45 pow, powf—x to the power y

Synopsis

```
#include <math.h>
double pow(double x, double y);
float powf(float x, float y);
```

Description

`pow` and `powf` calculate x raised to the exponent y. (That is, x^y.)

Returns

On success, `pow` and `powf` return the value calculated.

When the argument values would produce overflow, `pow` returns `HUGE_VAL` and set `errno` to `ERANGE`. If the argument x passed to `pow` or `powf` is a negative noninteger, and y is also not an integer, then `errno` is set to `EDOM`. If x and y are both 0, then `pow` and `powf` return `1`.

Portability

`pow` is ANSI C. `powf` is an extension.

1.46 pow10, pow10f—base 10 power functions

Synopsis

```
#include <math.h>
double pow10(double x);
float pow10f(float x);
```

Description

pow10 and pow10f calculate 10 ^ x, that is, 10^x

Returns

On success, pow10 and pow10f return the calculated value. If the result underflows, the returned value is 0. If the result overflows, the returned value is HUGE_VAL. In either case, errno is set to ERANGE.

Portability

pow10 and pow10f are GNU extensions.

1.47 `remainder`, `remainderf`—round and remainder

Synopsis

```
#include <math.h>
double remainder(double x, double y);
float remainderf(float x, float y);
```

Description

`remainder` and `remainderf` find the remainder of x/y; this value is in the range -$y/2$ .. +$y/2$.

Returns

`remainder` returns the integer result as a double.

Portability

`remainder` is a System V release 4. `remainderf` is an extension.

1.48 remquo, remquof—remainder and part of quotient

Synopsis

```
#include <math.h>
double remquo(double x, double y, int *quo);
float remquof(float x, float y, int *quo);
```

Description

The remquo functions compute the same remainder as the remainder functions; this value is in the range -*y*/2 ... +*y*/2. In the object pointed to by quo they store a value whose sign is the sign of x/y and whose magnitude is congruent modulo 2**n to the magnitude of the integral quotient of x/y. (That is, quo is given the n lsbs of the quotient, not counting the sign.) This implementation uses n=31 if int is 32 bits or more, otherwise, n is 1 less than the width of int.

For example:

```
remquo(-29.0, 3.0, &quo)
```

returns -1.0 and sets *quo*=10, and

```
remquo(-98307.0, 3.0, &quo)
```

returns -0.0 and sets *quo*=-32769, although for 16-bit int, *quo*=-1. In the latter case, the actual quotient of -(32769=0x8001) is reduced to -1 because of the 15-bit limitation for the quotient.

Returns

When either argument is NaN, NaN is returned. If *y* is 0 or *x* is infinite (and neither is NaN), a domain error occurs (i.e. the "invalid" floating point exception is raised or errno is set to EDOM), and NaN is returned. Otherwise, the remquo functions return *x* REM *y*.

Bugs

IEEE754-2008 calls for remquo(subnormal, inf) to cause the "underflow" floating-point exception. This implementation does not.

Portability

C99, POSIX.

1.49 `rint`, `rintf`—round to integer

Synopsis

```
#include <math.h>
double rint(double x);
float rintf(float x);
```

Description
The `rint` functions round their argument to an integer value in floating-point format, using the current rounding direction. They raise the "inexact" floating-point exception if the result differs in value from the argument. See the `nearbyint` functions for the same function with the "inexact" floating-point exception never being raised. Newlib does not directly support floating-point exceptions. The `rint` functions are written so that the "inexact" exception is raised in hardware implementations that support it, even though Newlib does not provide access.

Returns
x rounded to an integral value, using the current rounding direction.

Portability
ANSI C, POSIX

See Also
`nearbyint`, `round`

1.50 `round`, `roundf`—round to integer, to nearest

Synopsis

```
#include <math.h>
double round(double x);
float roundf(float x);
```

Description

The `round` functions round their argument to the nearest integer value in floating-point format, rounding halfway cases away from zero, regardless of the current rounding direction. (While the "inexact" floating-point exception behavior is unspecified by the C standard, the `round` functions are written so that "inexact" is not raised if the result does not equal the argument, which behavior is as recommended by IEEE 754 for its related functions.)

Returns

x rounded to an integral value.

Portability

ANSI C, POSIX

See Also

`nearbyint`, `rint`

1.51 scalbn, scalbnf, scalbln, scalblnf—scale by power of FLT_RADIX (=2)

Synopsis

```
#include <math.h>
double scalbn(double x, int n);
float scalbnf(float x, int n);
double scalbln(double x, long int n);
float scalblnf(float x, long int n);
```

Description

The `scalbn` and `scalbln` functions compute $x \cdot FLT_RADIX^n$. efficiently. The result is computed by manipulating the exponent, rather than by actually performing an exponentiation or multiplication. In this floating-point implementation FLT_RADIX=2, which makes the `scalbn` functions equivalent to the `ldexp` functions.

Returns

x times 2 to the power n. A range error may occur.

Portability

ANSI C, POSIX

See Also

`ldexp`

1.52 signbit—Does floating-point number have negative sign?

Synopsis

```
#include <math.h>
int signbit(real-floating x);
```

Description

The signbit macro determines whether the sign of its argument value is negative. The macro reports the sign of all values, including infinities, zeros, and NaNs. If zero is unsigned, it is treated as positive. As shown in the synopsis, the argument is "real-floating," meaning that any of the real floating-point types (float, double, etc.) may be given to it.

Note that because of the possibilities of signed 0 and NaNs, the expression "$x < 0.0$" does not give the same result as signbit in all cases.

Returns

The signbit macro returns a nonzero value if and only if the sign of its argument value is negative.

Portability

C99, POSIX.

1.53 `sin`, `sinf`, `cos`, `cosf`—sine or cosine

Synopsis

```
#include <math.h>
double sin(double x);
float  sinf(float x);
double cos(double x);
float cosf(float x);
```

Description

`sin` and `cos` compute (respectively) the sine and cosine of the argument x. Angles are specified in radians.

`sinf` and `cosf` are identical, save that they take and return `float` values.

Returns

The sine or cosine of x is returned.

Portability

`sin` and `cos` are ANSI C. `sinf` and `cosf` are extensions.

1.54 sinh, sinhf—hyperbolic sine

Synopsis

```
#include <math.h>
double sinh(double x);
float  sinhf(float x);
```

Description

sinh computes the hyperbolic sine of the argument x. Angles are specified in radians. sinh(x) is defined as

$$\frac{e^x - e^{-x}}{2}$$

sinhf is identical, save that it takes and returns float values.

Returns

The hyperbolic sine of x is returned.

When the correct result is too large to be representable (an overflow), sinh returns HUGE_VAL with the appropriate sign, and sets the global value errno to ERANGE.

Portability

sinh is ANSI C. sinhf is an extension.

1.55 `sqrt`, `sqrtf`—positive square root

Synopsis

```
#include <math.h>
double sqrt(double x);
float  sqrtf(float x);
```

Description

`sqrt` computes the positive square root of the argument.

Returns

On success, the square root is returned. If x is real and positive, then the result is positive. If x is real and negative, the global value `errno` is set to `EDOM` (domain error).

Portability

`sqrt` is ANSI C. `sqrtf` is an extension.

1.56 `tan`, `tanf`—tangent

Synopsis

```
#include <math.h>
double tan(double x);
float tanf(float x);
```

Description

`tan` computes the tangent of the argument x. Angles are specified in radians.

`tanf` is identical, save that it takes and returns `float` values.

Returns

The tangent of x is returned.

Portability

`tan` is ANSI. `tanf` is an extension.

1.57 `tanh`, `tanhf`—hyperbolic tangent

Synopsis

```
#include <math.h>
double tanh(double x);
float tanhf(float x);
```

Description

`tanh` computes the hyperbolic tangent of the argument x. Angles are specified in radians.

`tanh(x)` is defined as

```
sinh(x)/cosh(x)
```

`tanhf` is identical, save that it takes and returns `float` values.

Returns

The hyperbolic tangent of x is returned.

Portability

`tanh` is ANSI C. `tanhf` is an extension.

1.58 `trunc`, `truncf`—round to integer, towards zero

Synopsis

```
#include <math.h>
double trunc(double x);
float truncf(float x);
```

Description

The `trunc` functions round their argument to the integer value, in floating format, nearest to but no larger in magnitude than the argument, regardless of the current rounding direction. (While the "inexact" floating-point exception behavior is unspecified by the C standard, the `trunc` functions are written so that "inexact" is not raised if the result does not equal the argument, which behavior is as recommended by IEEE 754 for its related functions.)

Returns

x truncated to an integral value.

Portability

ANSI C, POSIX

2 Mathematical Complex Functions (`complex.h`)

This chapter groups the complex mathematical functions. The corresponding definitions and declarations are in `complex.h`. Functions and documentations are taken from NetBSD.

2.1 `cabs`, `cabsf`, `cabsl`—complex absolute-value

Synopsis

```
#include <complex.h>
double cabs(double complex z);
float cabsf(float complex z);
long double cabsl(long double complex z);
```

Description

These functions compute compute the complex absolute value (also called norm, modulus, or magnitude) of *z*.

`cabsf` is identical to `cabs`, except that it performs its calculations on `float complex`.

`cabsl` is identical to `cabs`, except that it performs its calculations on `long double complex`.

Returns

The cabs* functions return the complex absolute value.

Portability

`cabs`, `cabsf` and `cabsl` are ISO C99

2.2 cacos, cacosf—complex arc cosine

Synopsis

```
#include <complex.h>
double complex cacos(double complex z);
float complex cacosf(float complex z);
```

Description

These functions compute the complex arc cosine of z, with branch cuts outside the interval [-1, +1] along the real axis.

`cacosf` is identical to `cacos`, except that it performs its calculations on `floats complex`.

Returns

These functions return the complex arc cosine value, in the range of a strip mathematically unbounded along the imaginary axis and in the interval [0, π] along the real axis.

Portability

`cacos` and `cacosf` are ISO C99

2.3 cacosh, cacoshf—complex arc hyperbolic cosine

Synopsis

```
#include <complex.h>
double complex cacosh(double complex z);
float complex cacoshf(float complex z);
```

Description

These functions compute the complex arc hyperbolic cosine of z, with a branch cut at values less than 1 along the real axis.

`cacoshf` is identical to `cacosh`, except that it performs its calculations on `floats complex`.

Returns

These functions return the complex arc hyperbolic cosine value, in the range of a half-strip of non-negative values along the real axis and in the interval $[-i\pi, +i\pi]$ along the imaginary axis.

Portability

`cacosh` and `cacoshf` are ISO C99

2.4 carg, cargf—argument (phase angle)

Synopsis

```
#include <complex.h>
double carg(double complex z);
float cargf(float complex z);
```

Description

These functions compute the argument (also called phase angle) of z, with a branch cut along the negative real axis.

`cargf` is identical to `carg`, except that it performs its calculations on `floats complex`.

Returns

The carg functions return the value of the argument in the interval $[-\pi, +\pi]$

Portability

`carg` and `cargf` are ISO C99

2.5 `casin`, `casinf`—complex arc sine

Synopsis

```
#include <complex.h>
double complex casin(double complex z);
float complex casinf(float complex z);
```

Description

These functions compute the complex arc sine of z, with branch cuts outside the interval [-1, +1] along the real axis.

`casinf` is identical to `casin`, except that it performs its calculations on `floats complex`.

Returns

These functions return the complex arc sine value, in the range of a strip mathematically unbounded along the imaginary axis and in the interval $[-\pi/2, +\pi/2]$ along the real axis.

Portability

`casin` and `casinf` are ISO C99

2.6 casinh, casinhf—complex arc hyperbolic sine

Synopsis

```
#include <complex.h>
double complex casinh(double complex z);
float complex casinhf(float complex z);
```

Description

These functions compute the complex arc hyperbolic sine of z, with branch cuts outside the interval $[-i, +i]$ along the imaginary axis.

`casinhf` is identical to `casinh`, except that it performs its calculations on `floats complex`.

Returns

These functions return the complex arc hyperbolic sine value, in the range of a strip mathematically unbounded along the real axis and in the interval $[-i\pi/2, +i\pi/2]$ along the imaginary axis.

Portability

`casinh` and `casinhf` are ISO C99

2.7 `catan`, `catanf`—complex arc tangent

Synopsis

```
#include <complex.h>
double complex catan(double complex z);
float complex catanf(float complex z);
```

Description

These functions compute the complex arc tangent of z, with branch cuts outside the interval $[-i, +i]$ along the imaginary axis.

`catanf` is identical to `catan`, except that it performs its calculations on `floats complex`.

Returns

These functions return the complex arc tangent, in the range of a strip mathematically unbounded along the imaginary axis and in the interval $[-\pi/2, +\pi/2]$ along the real axis.

Portability

`catan` and `catanf` are ISO C99

2.8 catanh, catanhf—complex arc hyperbolic tangent

Synopsis

```
#include <complex.h>
double complex catanh(double complex z);
float complex catanhf(float complex z);
```

Description

These functions compute the complex arc hyperbolic tan of z, with branch cuts outside the interval [-1, +1] along the real axis.

`catanhf` is identical to `catanh`, except that it performs its calculations on `floats complex`.

Returns

These functions return the complex arc hyperbolic tangent value, in the range of a strip mathematically unbounded along the real axis and in the interval $[-i\pi/2, +i\pi/2]$ along the imaginary axis.

Portability

`catanh` and `catanhf` are ISO C99

2.9 ccos, ccosf—complex cosine

Synopsis

```
#include <complex.h>
double complex ccos(double complex z);
float complex ccosf(float complex z);
```

Description

These functions compute the complex cosine of z.

`ccosf` is identical to `ccos`, except that it performs its calculations on `floats complex`.

Returns

These functions return the complex cosine value.

Portability

`ccos` and `ccosf` are ISO C99

2.10 ccosh, ccoshf—complex hyperbolic cosine

Synopsis

```
#include <complex.h>
double complex ccosh(double complex z);
float complex ccoshf(float complex z);
```

Description

These functions compute the complex hyperbolic cosine of z.

`ccoshf` is identical to `ccosh`, except that it performs its calculations on `floats complex`.

Returns

These functions return the complex hyperbolic cosine value.

Portability

`ccosh` and `ccoshf` are ISO C99

2.11 `cexp`, `cexpf`—complex base-e exponential

Synopsis

```
#include <complex.h>
double complex cexp(double complex z);
float complex cexpf(float complex z);
```

Description

These functions compute the complex base-e exponential of z.

`cexpf` is identical to `cexp`, except that it performs its calculations on `floats complex`.

Returns

The cexp functions return the complex base-e exponential value.

Portability

`cexp` and `cexpf` are ISO C99

2.12 cimag, cimagf, cimagl—imaginary part

Synopsis

```
#include <complex.h>
double cimag(double complex z);
float cimagf(float complex z);
long double cimagl(long double complex z);
```

Description

These functions compute the imaginary part of *z*.

`cimagf` is identical to `cimag`, except that it performs its calculations on `float complex`.

`cimagl` is identical to `cimag`, except that it performs its calculations on `long double complex`.

Returns

The cimag* functions return the imaginary part value (as a real).

Portability

`cimag`, `cimagf` and `cimagl` are ISO C99

2.13 `clog`, `clogf`—complex base-e logarithm

Synopsis

```
#include <complex.h>
double complex clog(double complex z);
float complex clogf(float complex z);
```

Description

These functions compute the complex natural (base-e) logarithm of z, with a branch cut along the negative real axis.

`clogf` is identical to `clog`, except that it performs its calculations on `floats complex`.

Returns

The clog functions return the complex natural logarithm value, in the range of a strip mathematically unbounded along the real axis and in the interval $[-i\pi, +i\pi]$ along the imaginary axis.

Portability

`clog` and `clogf` are ISO C99

2.14 clog10, clog10f—complex base-10 logarithm

Synopsis

```
#define _GNU_SOURCE
#include <complex.h>
double complex clog10(double complex z);
float complex clog10f(float complex z);
```

Description

These functions compute the complex base-10 logarithm of z. `clog10` is equivalent to `clog`(z)/`log`(10).

`clog10f` is identical to `clog10`, except that it performs its calculations on `floats complex`.

Returns

The clog10 functions return the complex base-10 logarithm value.

Portability

`clog10` and `clog10f` are GNU extensions.

2.15 `conj`, `conjf`—complex conjugate

Synopsis

```
#include <complex.h>
double complex conj(double complex z);
float complex conjf(float complex z);
```

Description

These functions compute the complex conjugate of z, by reversing the sign of its imaginary part.

`conjf` is identical to `conj`, except that it performs its calculations on `floats complex`.

Returns

The conj functions return the complex conjugate value.

Portability

`conj` and `conjf` are ISO C99

2.16 cpow, cpowf—complex power

Synopsis

```
#include <complex.h>
double complex cpow(double complex x, double complex y);
float complex cpowf(float complex x, float complex y);
```

Description

The cpow functions compute the complex power function x^y power, with a branch cut for the first parameter along the negative real axis.

`cpowf` is identical to `cpow`, except that it performs its calculations on `floats complex`.

Returns

The cpow functions return the complex power function value.

Portability

`cpow` and `cpowf` are ISO C99

2.17 `cproj`, `cprojf`— Riemann sphere projection

Synopsis

```
#include <complex.h>
double complex cproj(double complex z);
float complex cprojf(float complex z);
```

Description

These functions compute a projection of z onto the Riemann sphere: z projects to z except that all complex infinities (even those with one infinite part and one NaN part) project to positive infinity on the real axis. If z has an infinite part, then `cproj`(z) is equivalent to

INFINITY + I * copysign(0.0, cimag(z))

`cprojf` is identical to `cproj`, except that it performs its calculations on `floats complex`.

Returns

The cproj functions return the value of the projection onto the Riemann sphere.

Portability

`cproj` and `cprojf` are ISO C99

2.18 creal, crealf, creall—real part

Synopsis

```
#include <complex.h>
double creal(double complex z);
float crealf(float complex z);
double long creall(long double complex z);
```

Description

These functions compute the real part of *z*.

`crealf` is identical to `creal`, except that it performs its calculations on `float complex`.

`creall` is identical to `creal`, except that it performs its calculations on `long double complex`.

Returns

The creal* functions return the real part value.

Portability

`creal`, `crealf` and `creall` are ISO C99

2.19 `csin`, `csinf`—complex sine

Synopsis

```
#include <complex.h>
double complex csin(double complex z);
float complex csinf(float complex z);
```

Description

These functions compute the complex sine of z.

`csinf` is identical to `csin`, except that it performs its calculations on `floats complex`.

Returns

These functions return the complex sine value.

Portability

`csin` and `csinf` are ISO C99

2.20 csinh, csinhf—complex hyperbolic sine

Synopsis

```
#include <complex.h>
double complex csinh(double complex z);
float complex csinhf(float complex z);
```

Description

These functions compute the complex hyperbolic sine of z.

`ccoshf` is identical to `ccosh`, except that it performs its calculations on `floats complex`.

Returns

These functions return the complex hyperbolic sine value.

Portability

`csinh` and `csinhf` are ISO C99

2.21 `csqrt`, `csqrtf`—complex square root

Synopsis

```
#include <complex.h>
double complex csqrt(double complex z);
float complex csqrtf(float complex z);
```

Description

These functions compute the complex square root of z, with a branch cut along the negative real axis.

`csqrtf` is identical to `csqrt`, except that it performs its calculations on `floats complex`.

Returns

The csqrt functions return the complex square root value, in the range of the right halfplane (including the imaginary axis).

Portability

`csqrt` and `csqrtf` are ISO C99

2.22 ctan, ctanf—complex tangent

Synopsis

```
#include <complex.h>
double complex ctan(double complex z);
float complex ctanf(float complex z);
```

Description

These functions compute the complex tangent of z.

`ctanf` is identical to `ctan`, except that it performs its calculations on `floats complex`.

Returns

These functions return the complex tangent value.

Portability

`ctan` and `ctanf` are ISO C99

2.23 `ctanh`, `ctanf`—complex hyperbolic tangent

Synopsis

```
#include <complex.h>
double complex ctanh(double complex z);
float complex ctanhf(float complex z);
```

Description

These functions compute the complex hyperbolic tangent of z.

`ctanhf` is identical to `ctanh`, except that it performs its calculations on `floats complex`.

Returns

These functions return the complex hyperbolic tangent value.

Portability

`ctanh` and `ctanhf` are ISO C99

3 Floating-Point Environment (`fenv.h`)

This chapter groups the methods used to manipulate the floating-point status flags. Floating-point operations modify the floating-point status flags to indicate abnormal result information.

The implementation of these methods is architecture specific.

3.1 `feclearexcept`—clear floating-point exception

Synopsis

```
#include <fenv.h>
int feclearexcept(int except);

Link with -lm.
```

Description

This method attempts to clear the floating-point exceptions specified in *except*.

Returns

If the *except* argument is zero or all requested exceptions were successfully cleared, this method returns zero. Otherwise, a non-zero value is returned.

Portability

ANSI C requires `feclearexcept`.

Not all Newlib targets have a working implementation. Refer to the file `sys/fenv.h` to see the status for your target.

3.2 fegetenv—get current floating-point environment

Synopsis

```
#include <fenv.h>
int fegetenv(fenv_t *envp);

Link with -lm.
```

Description

This method attempts to return the floating-point environment in the area specified by *envp*.

Returns

If floating-point environment was successfully returned, then this method returns zero. Otherwise, a non-zero value is returned.

Portability

ANSI C requires `fegetenv`.

Not all Newlib targets have a working implementation. Refer to the file `sys/fenv.h` to see the status for your target.

3.3 `fegetexceptflag`—get floating-point status flags

Synopsis

```
#include <fenv.h>
int fegetexceptflag(fexcept_t *flagp, int excepts);

Link with -lm.
```

Description

This method attempts to store an implementation-defined representation of the states of the floating-point status flags specified by *excepts* in the memory pointed to by *flagp*.

Returns

If the information was successfully returned, this method returns zero. Otherwise, a non-zero value is returned.

Portability

ANSI C requires `fegetexceptflag`.

Not all Newlib targets have a working implementation. Refer to the file `sys/fenv.h` to see the status for your target.

3.4 fegetround—get current rounding direction

Synopsis

```
#include <fenv.h>
int fegetround(void);

Link with -lm.
```

Description

This method returns the current rounding direction.

Returns

This method returns the rounding direction, corresponding to the value of the respective rouding macro. If the current rounding direction cannot be determined, then a negative value is returned.

Portability

ANSI C requires `fegetround`.

Not all Newlib targets have a working implementation. Refer to the file `sys/fenv.h` to see the status for your target.

3.5 `feholdexcept`—save current floating-point environment

Synopsis

```
#include <fenv.h>
int feholdexcept(fenv_t *envp);

Link with -lm.
```

Description

This method attempts to save the current floating-point environment in the fenv_t instance pointed to by *envp*, clear the floating point status flags, and then, if supported by the target architecture, install a "non-stop" (e.g. continue on floating point exceptions) mode for all floating-point exceptions.

Returns

This method will return zero if the non-stop floating-point exception handler was installed. Otherwise, a non-zero value is returned.

Portability

ANSI C requires `feholdexcept`.

Not all Newlib targets have a working implementation. Refer to the file `sys/fenv.h` to see the status for your target.

3.6 feraiseexcept—raise floating-point exception

Synopsis

```
#include <fenv.h>
int feraiseexcept(int excepts);

Link with -lm.
```

Description

This method attempts to raise the floating-point exceptions specified in *excepts*.

Returns

If the *excepts* argument is zero or all requested exceptions were successfully raised, this method returns zero. Otherwise, a non-zero value is returned.

Portability

ANSI C requires `feraiseexcept`.

Not all Newlib targets have a working implementation. Refer to the file `sys/fenv.h` to see the status for your target.

3.7 `fesetenv`—set current floating-point environment

Synopsis

```
#include <fenv.h>
int fesetenv(const fenv_t *envp);

Link with -lm.
```

Description

This method attempts to establish the floating-point environment pointed to by *envp*. The argument *envp* must point to a floating-point environment obtained via `fegetenv` or `feholdexcept` or a floating-point environment macro such as `FE_DFL_ENV`.

It only sets the states of the flags as recorded in its argument, and does not actually raise the associated floating-point exceptions.

Returns

If floating-point environment was successfully established, then this method returns zero. Otherwise, a non-zero value is returned.

Portability

ANSI C requires `fesetenv`.

Not all Newlib targets have a working implementation. Refer to the file `sys/fenv.h` to see the status for your target.

3.8 fesetexceptflag—set floating-point status flags

Synopsis

```
#include <fenv.h>
int fesetexceptflag(const fexcept_t *flagp, int excepts);

Link with -lm.
```

Description

This method attempts to set the floating-point status flags specified by *excepts* to the states indicated by *flagp*. The argument *flagp* must point to an fexcept_t instance obtained via calling `fegetexceptflag` with at least the floating-point exceptions specified by the argument *excepts*.

This method does not raise any floating-point exceptions. It only sets the state of the flags.

Returns

If the information was successfully returned, this method returns zero. Otherwise, a non-zero value is returned.

Portability

ANSI C requires `fesetexceptflag`.

Not all Newlib targets have a working implementation. Refer to the file `sys/fenv.h` to see the status for your target.

3.9 `fesetround`—set current rounding direction

Synopsis

```
#include <fenv.h>
int fesetround(int round);

Link with -lm.
```

Description

This method attempts to set the current rounding direction represented by *round*. *round* must be the value of one of the rounding-direction macros.

Returns

If the rounding mode was successfully established, this method returns zero. Otherwise, a non-zero value is returned.

Portability

ANSI C requires `fesetround`.

Not all Newlib targets have a working implementation. Refer to the file `sys/fenv.h` to see the status for your target.

3.10 fetestexcept—test floating-point exception flags

Synopsis

```
#include <fenv.h>
int fetestexcept(int except);

Link with -lm.
```

Description

This method test the current floating-point exceptions to determine which of those specified in *except* are currently set.

Returns

This method returns the bitwise-inclusive OR of the floating point exception macros which correspond to the currently set floating point exceptions.

Portability

ANSI C requires `fetestexcept`.

Not all Newlib targets have a working implementation. Refer to the file `sys/fenv.h` to see the status for your target.

3.11 `feupdateenv`—update current floating-point environment

Synopsis

```
#include <fenv.h>
int feupdateenv(const fenv_t *envp);

Link with -lm.
```

Description

This method attempts to save the currently raised floating point exceptions in its automatic storage, install the floating point environment specified by *envp*, and raise the saved floating point exceptions.

The argument *envp* must point to a floating-point environment obtained via `fegetenv` or `feholdexcept`.

Returns

If all actions are completed successfully, then this method returns zero. Otherwise, a non-zero value is returned.

Portability

ANSI C requires `feupdateenv`.

Not all Newlib targets have a working implementation. Refer to the file `sys/fenv.h` to see the status for your target.

4 Reentrancy Properties of `libm`

When a libm function detects an exceptional case, `errno` may be set.

`errno` is a macro which expands to the per-thread error value. This makes it thread safe, and therefore reentrant.

5 The long double function support of `libm`

Currently, the full set of long double math functions is only provided on platforms where long double equals double. For such platforms, the long double math functions are implemented as calls to the double versions.

Document Index

M

`matherr` and reentrancy 99

R

reentrancy 99

The body of this manual is set in
cmr10,
with headings in **cmbx10**
and examples in `cmtt10`.
cmti10 and
cmsl10
are used for emphasis.

Table of Contents

1 Mathematical Functions (`math.h`) 1

1.1 Error Handling 2
1.2 Standards Compliance And Portability 2
1.3 `acos`, `acosf`—arc cosine 3
1.4 `acosh`, `acoshf`—inverse hyperbolic cosine 4
1.5 `asin`, `asinf`—arc sine 5
1.6 `asinh`, `asinhf`—inverse hyperbolic sine 6
1.7 `atan`, `atanf`—arc tangent 7
1.8 `atan2`, `atan2f`—arc tangent of y/x 8
1.9 `atanh`, `atanhf`—inverse hyperbolic tangent 9
1.10 `jN`, `jNf`, `yN`, `yNf`—Bessel functions 10
1.11 `cbrt`, `cbrtf`—cube root 11
1.12 `copysign`, `copysignf`—sign of *y*, magnitude of *x* 12
1.13 `cosh`, `coshf`—hyperbolic cosine 13
1.14 `erf`, `erff`, `erfc`, `erfcf`—error function 14
1.15 `exp`, `expf`—exponential 15
1.16 `exp10`, `exp10f`—exponential, base 10 16
1.17 `exp2`, `exp2f`—exponential, base 2 17
1.18 `expm1`, `expm1f`—exponential minus 1 18
1.19 `fabs`, `fabsf`—absolute value (magnitude) 19
1.20 `fdim`, `fdimf`—positive difference 20
1.21 `floor`, `floorf`, `ceil`, `ceilf`—floor and ceiling 21
1.22 `fma`, `fmaf`—floating multiply add 22
1.23 `fmax`, `fmaxf`—maximum 23
1.24 `fmin`, `fminf`—minimum 24
1.25 `fmod`, `fmodf`—floating-point remainder (modulo) 25
1.26 `frexp`, `frexpf`—split floating-point number 26
1.27 `gamma`, `gammaf`, `lgamma`, `lgammaf`, `gamma_r`, `gammaf_r`, `lgamma_r`, `lgammaf_r`, `tgamma`, and `tgammaf`—logarithmic and plain gamma functions 27
1.28 `hypot`, `hypotf`—distance from origin 29
1.29 `ilogb`, `ilogbf`—get exponent of floating-point number 30
1.30 `infinity`, `infinityf`—representation of infinity 31
1.31 `isgreater`, `isgreaterequal`, `isless`, `islessequal`, `islessgreater`, and `isunordered`—comparison macros 32
1.32 `fpclassify`, `isfinite`, `isinf`, `isnan`, and `isnormal`—floating-point classification macros; `finite`, `finitef`, `isinf`, `isinff`, `isnan`, `isnanf`—test for exceptional numbers 34
1.33 `ldexp`, `ldexpf`—load exponent 36
1.34 `log`, `logf`—natural logarithms 37
1.35 `log10`, `log10f`—base 10 logarithms 38
1.36 `log1p`, `log1pf`—log of `1 + x` 39
1.37 `log2`, `log2f`—base 2 logarithm 40
1.38 `logb`, `logbf`—get exponent of floating-point number 41

1.39 lrint, lrintf, llrint, llrintf—round to integer 42
1.40 lround, lroundf, llround, llroundf—round to integer, to nearest 43
1.41 modf, modff—split fractional and integer parts 44
1.42 nan, nanf—representation of "Not a Number" 45
1.43 nearbyint, nearbyintf—round to integer 46
1.44 nextafter, nextafterf—get next number 47
1.45 pow, powf—x to the power y 48
1.46 pow10, pow10f—base 10 power functions 49
1.47 remainder, remainderf—round and remainder 50
1.48 remquo, remquof—remainder and part of quotient 51
1.49 rint, rintf—round to integer 52
1.50 round, roundf—round to integer, to nearest 53
1.51 scalbn, scalbnf, scalbln, scalblnf—scale by power of FLT_RADIX (=2) 54
1.52 signbit—Does floating-point number have negative sign? 55
1.53 sin, sinf, cos, cosf—sine or cosine 56
1.54 sinh, sinhf—hyperbolic sine 57
1.55 sqrt, sqrtf—positive square root 58
1.56 tan, tanf—tangent 59
1.57 tanh, tanhf—hyperbolic tangent 60
1.58 trunc, truncf—round to integer, towards zero 61

2 Mathematical Complex Functions (complex.h) .. 63

2.1 cabs, cabsf, cabsl—complex absolute-value 64
2.2 cacos, cacosf—complex arc cosine 65
2.3 cacosh, cacoshf—complex arc hyperbolic cosine 66
2.4 carg, cargf—argument (phase angle) 67
2.5 casin, casinf—complex arc sine 68
2.6 casinh, casinhf—complex arc hyperbolic sine 69
2.7 catan, catanf—complex arc tangent 70
2.8 catanh, catanhf—complex arc hyperbolic tangent 71
2.9 ccos, ccosf—complex cosine 72
2.10 ccosh, ccoshf—complex hyperbolic cosine 73
2.11 cexp, cexpf—complex base-e exponential 74
2.12 cimag, cimagf, cimagl—imaginary part 75
2.13 clog, clogf—complex base-e logarithm 76
2.14 clog10, clog10f—complex base-10 logarithm 77
2.15 conj, conjf—complex conjugate 78
2.16 cpow, cpowf—complex power 79
2.17 cproj, cprojf— Riemann sphere projection 80
2.18 creal, crealf, creall—real part 81
2.19 csin, csinf—complex sine 82
2.20 csinh, csinhf—complex hyperbolic sine 83
2.21 csqrt, csqrtf—complex square root 84
2.22 ctan, ctanf—complex tangent 85
2.23 ctanh, ctanf—complex hyperbolic tangent 86

3 Floating-Point Environment (`fenv.h`) 87

3.1 `feclearexcept`—clear floating-point exception 88
3.2 `fegetenv`—get current floating-point environment 89
3.3 `fegetexceptflag`—get floating-point status flags 90
3.4 `fegetround`—get current rounding direction 91
3.5 `feholdexcept`—save current floating-point environment 92
3.6 `feraiseexcept`—raise floating-point exception 93
3.7 `fesetenv`—set current floating-point environment 94
3.8 `fesetexceptflag`—set floating-point status flags 95
3.9 `fesetround`—set current rounding direction 96
3.10 `fetestexcept`—test floating-point exception flags 97
3.11 `feupdateenv`—update current floating-point environment 98

4 Reentrancy Properties of `libm` 99

5 The long double function support of `libm` ... 101

Document Index 103